Home task 2

1) write custom realization of method `bind` in javascript:

```
function bind(fn, context, args) {
  //...
}

const foo = () => {};
const context = {}; // any context, object, array, etc.
const data = []; // any object, array, etc.

const bindedFunction = bind(foo, context, data);
```

2) create an object with a magic property, and when any value is assigned to that property, current date will be output and value should be stored in the additional property *counter*. But if we read that property from the object, additional property counter will be incremented and output, ex.:

```
o.magicProperty = 5; // 'Sat Mar 24 2018 13:48:47 GMT+0300 (+03) -- 5'
console.log(o.magicProperty); // 6
console.log(o.magicProperty); // 7
console.log(o.magicProperty); // 8
```

tip.: read about getters and setters in objects

3) create a function-calculator which accepts two numbers and sign for calculation, but the signatuir of the function should looks like that:

```
calc(1, 2)('+'); // 3
calc(1, 2)('/'); // 0.5
```

4) create a function `curry`, which accepts any function with mathematical logic and the exact amout of arguments for further calls as it was declared in that function, ex.:

```
function sum2(x, y) { // 2 parameters
  return x + y;
}
function sum4(a, b, c, d) { // 4 parameters
  return a + b + c + d;
}

curry(sum2)(1)(2); // 2 calls, after first wrapping call, returns 3
curry(sum4)(2)(3)(4)(5); // 4 calls, after first wrapping call, returns 14
```

5) create a function, which can not be used as a constructor (can not be called with operator `new`)

6) create a function `sleep(seconds)`, which will work on the next way:

```
console.log(new Date()); // Sun Oct 08 2017 10:44:34 GMT+0300 (+03)
sleep(9);
console.log(new Date()); // Sun Oct 08 2017 10:44:43 GMT+0300 (+03)

// it means, that the second console.log will be triggered only after 9 seconds
```

7) Create a function `getCounter`, ex.:

```
var c = getCounter(5);
c
  .log() // 5 <- should outputs the result
  .add(4) // <- shoild add some value to the result
  .log() // 9
  .add(3)
  .log() // 12
  .reset() // <- resets the result
  .log() // 0
  .add(8)
  .log(); // 8
```